High performance computing

Processors

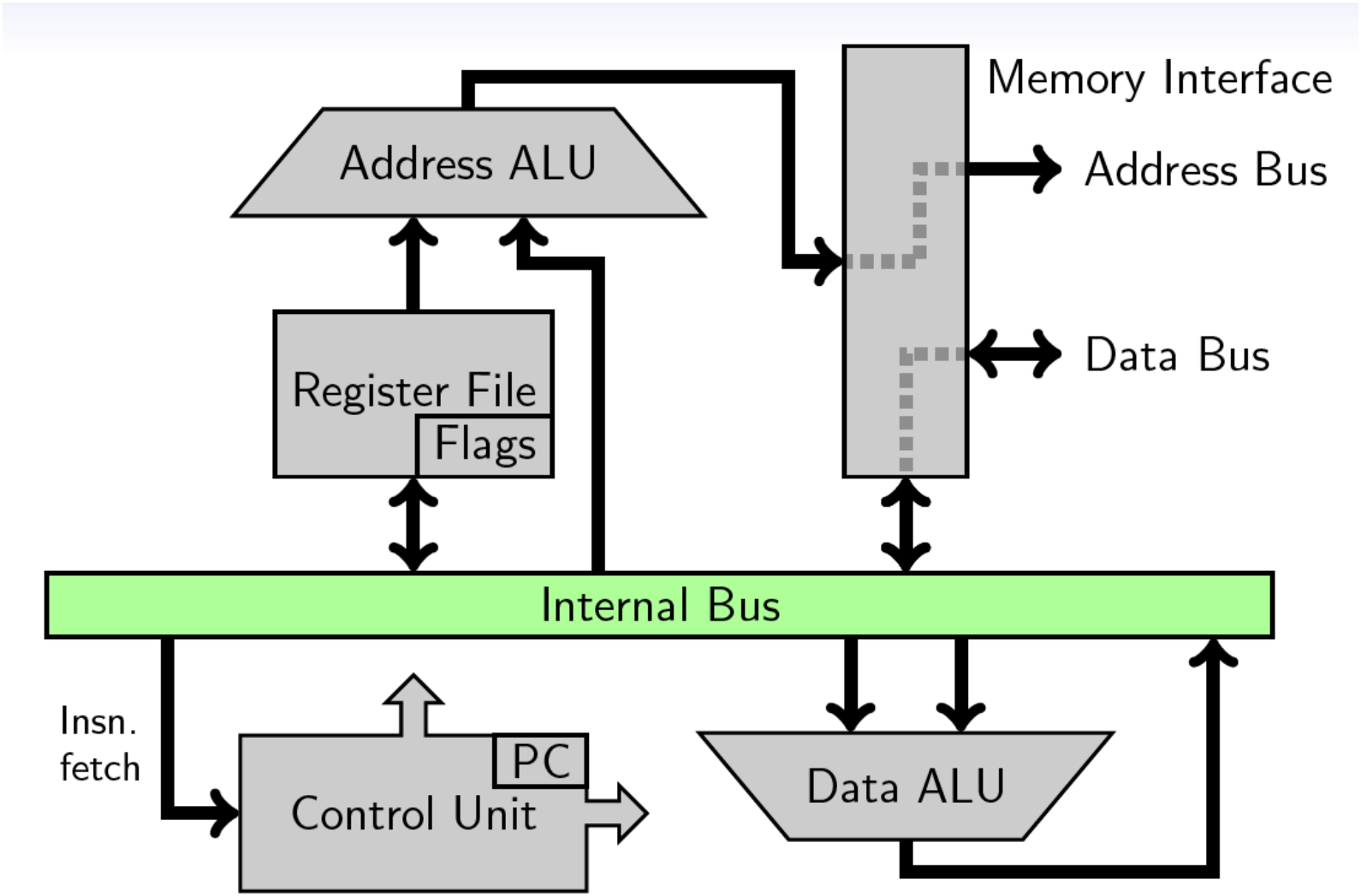
Performance of the computers

- Models of performance-> estimating the time for the task
- Performance - speed of assigned tasks execution, eg:
  - number of instructions per second
  - CPI (clocks per instruction) - the number of clock cycles needed for command execution
- An attempt to estimate the time:
  - which commands are executed in order to realize task
  - what is the duration of each of the commands (in the number of cycles - the CPI for each of the commands)
  - what is the frequency of the processor
  - multiply, summation - ready

Performance of the computers

- Basic parameters of hardware:
  - Clock Frequency
  - The patency of the processor - Instructions Per Cycle (IPC as the inverse of CPI)
  - Bandwidth of the memory-CPU bus
- Problem - why the parameters of the basic elements of the computer system don’t allow to estimate its performance in the implementation of practical tasks?
  - complexity of the processor architecture
  - complexity of system memory
  - cooperation of hardware, operating system and compilers, interpreters, virtual machines

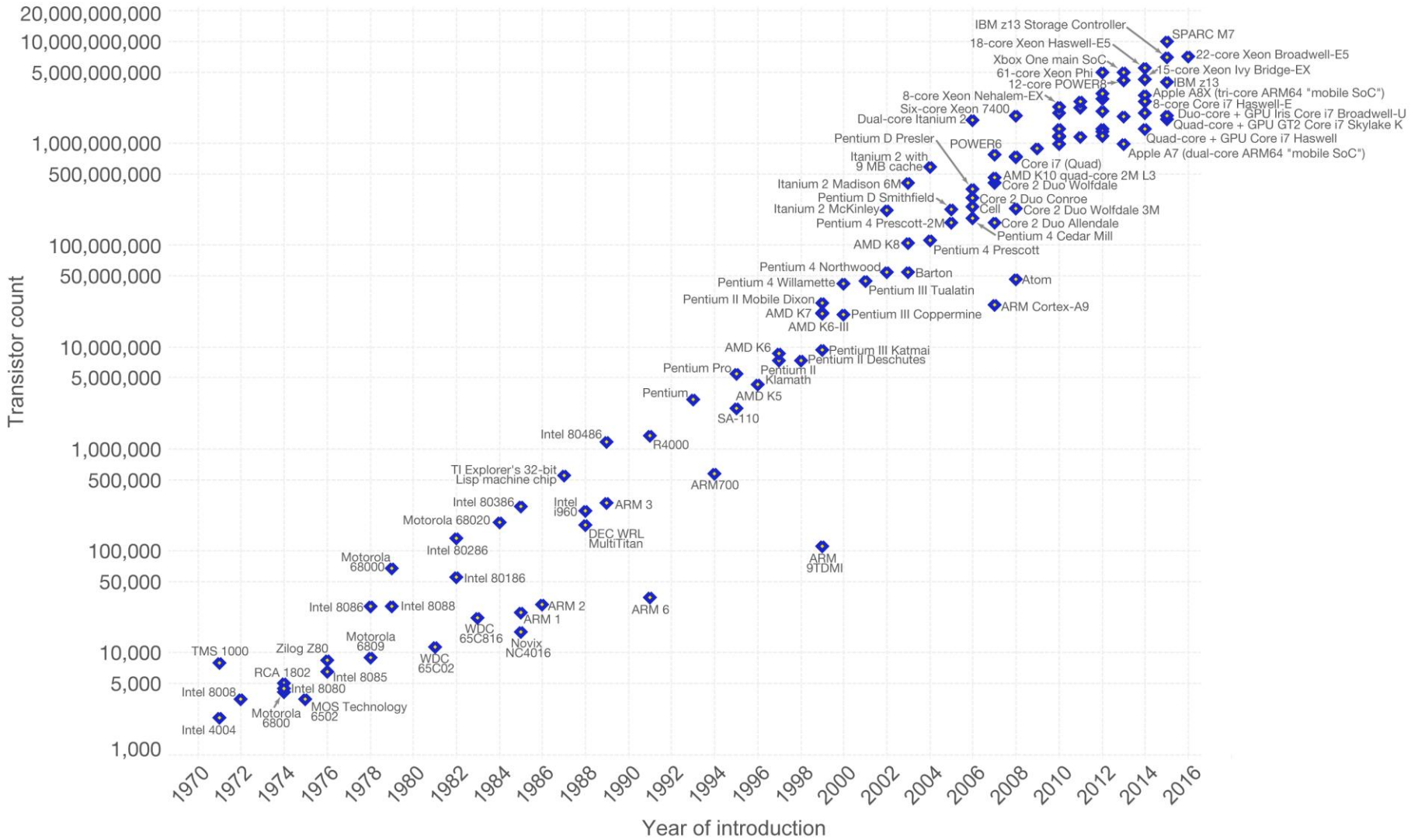
Von Neuman architecture



Moore's Law – The number of transistors on integrated circuit chips (1971-2016)

Our World in Data

Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important as other aspects of technological progress – such as processing speed or the price of electronic products – are strongly linked to Moore's law.



Data source: Wikipedia (https://en.wikipedia.org/wiki/Transistor_count)

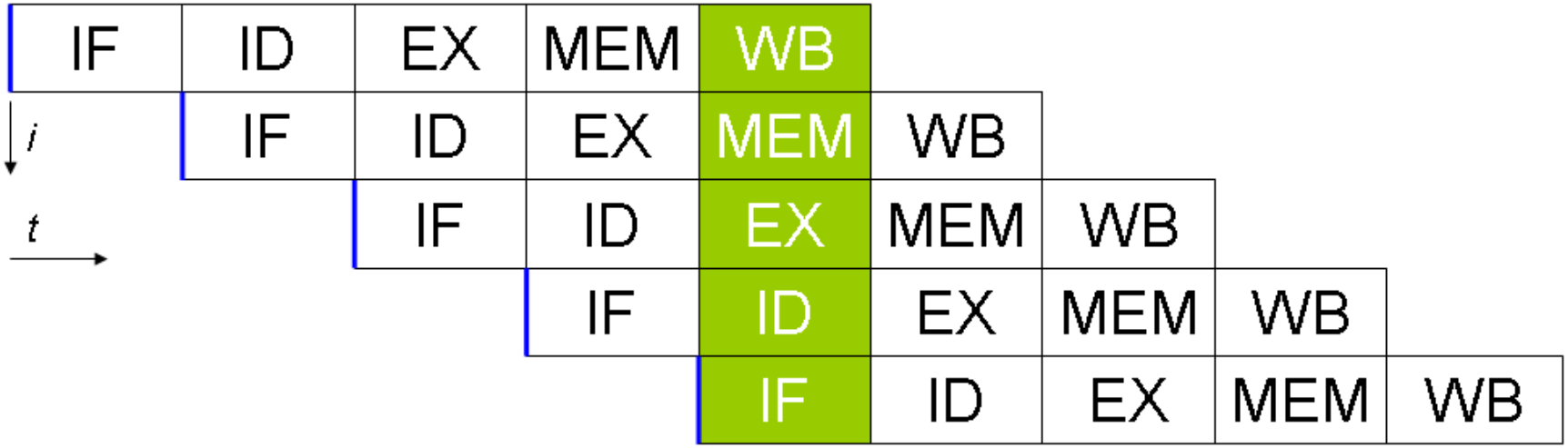
The data visualization is available at OurWorldinData.org. There you find more visualizations and research on this topic.

Licensed under CC-BY-SA by the author Max Roser.

Pipeline

- starting from the 60s
- an increase in the complexity of processors
- recognition of functional units in the processor
- division of the command execution
- typical steps in processor orders (in practice for a single order never occur all phases):
  - Calculate the address of the order, download an order, decode order (argument address calculation, load argument) - can be repeated, operation on arguments, result address calculation, save result
  - Official names: Instruction Fetch, Instruction Decode, Operand Fetch, Instruction Execution, Write Back.
  - Simplified description for further consideration: IF, ID, OF, IE, WB

Pipeline

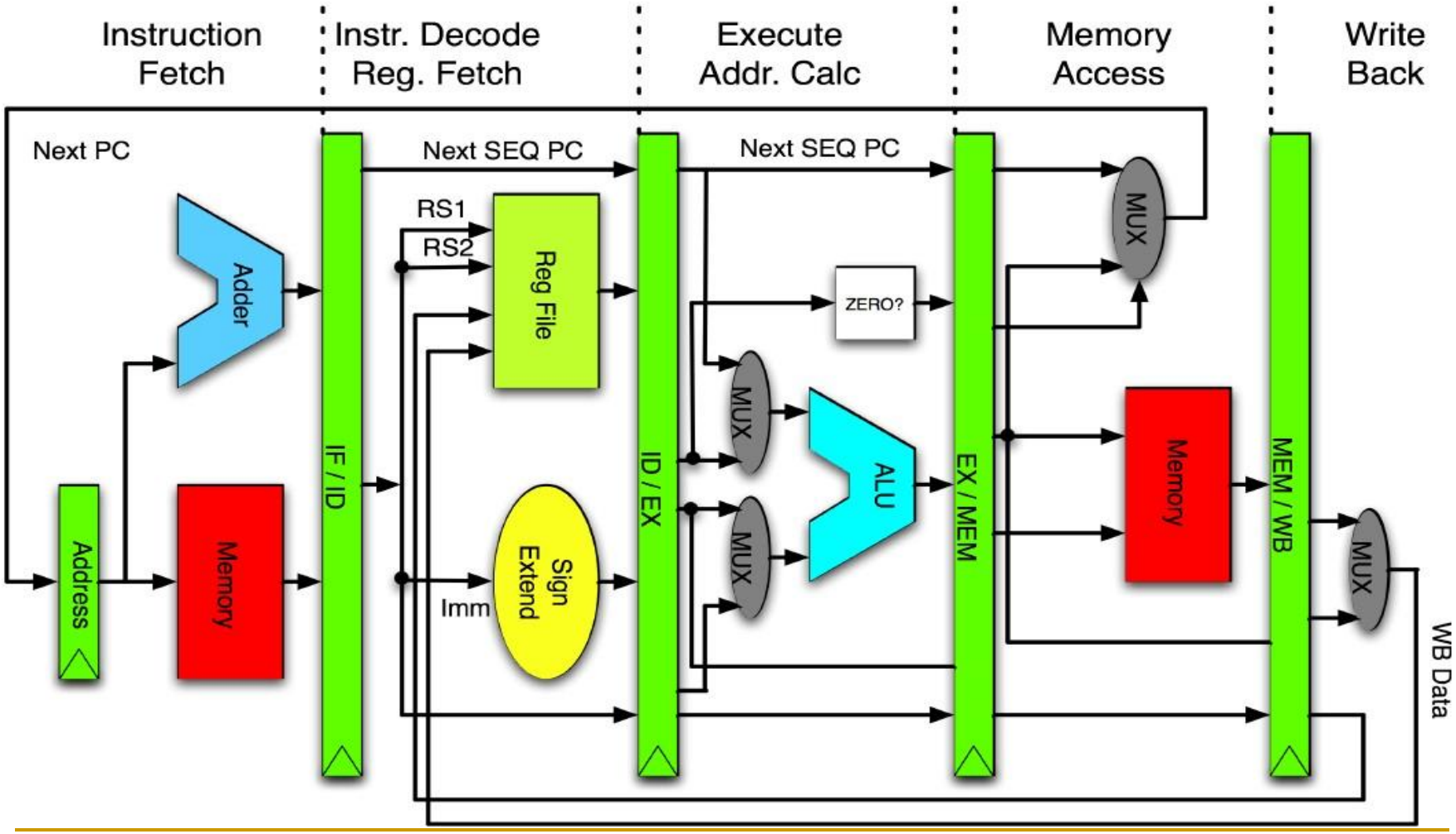

IF ID EX MEM WB
IF ID EX MEM WB
IF ID EX MEM WB
IF ID EX MEM WB
IF ID EX MEM WB
i
t

Pipelining

- Pipelining - increase the speed of operation:
  - Coarse analysis:
    - We assume the same time for processing order:

      the introduction of k-stage pipe increases the efficiency of k-fold (for a long enough sequence of commands)
  - Practical analysis
    - provided by the manufacturers of the hardware
      - maximum number of completed orders in each clock cycle (it can be less than one) IPC_{max}
      - clock frequency of the processor
  - Processors with a powerful streams may end one order in every cycle and have a much faster clock
  - Theoretical maximum CPU performance is the product of: patency (IPC_{max}) x clock frequency

Pipelining


Instruction Fetch
Instr. Decode Reg. Fetch
Execute Addr. Calc
Memory Access
Write Back
Next PC
Next SEQ PC
Next SEQ PC
Adder
Address
Memory
IF / ID
RS1
RS2
Reg File
Imm
Sign Extend
ID / EX
MUX
MUX
ZERO?
ALU
EX / MEM
MUX
Memory
MEM / WB
MUX
WB Data

Pipelining

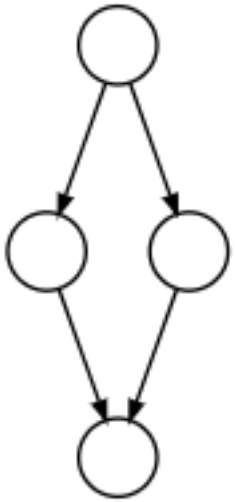
- Pipelining problems:
  - hazard: situations disrupting perfect pipelining (in other contexts to situations that disrupt processing the names: conflict and dependency are used)
    - resources hazard - processing of two orders requires access to a single resource at the same time
    - steering hazard - associated with jump orders
    - data hazard - related to the dependency between arguments of orders simultaneously processed (optimization techniques: renaming registers, transferring (forwarding) arguments)

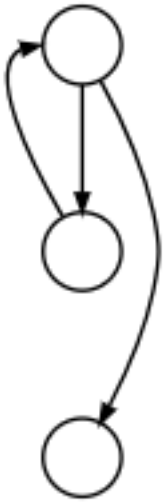
Steering hazards

- Statistics: jumps represent over 20% of orders
- Unconditional jumps:
  - avoiding downtime by prefetch
- Conditional jumps
  - branch delay slot
  - predict the result of branching:
    - jump never made
    - jump always made
    - static prediction (eg. on the basis of the direction of the jump)
    - dynamic prediction (based on the history of jumps)

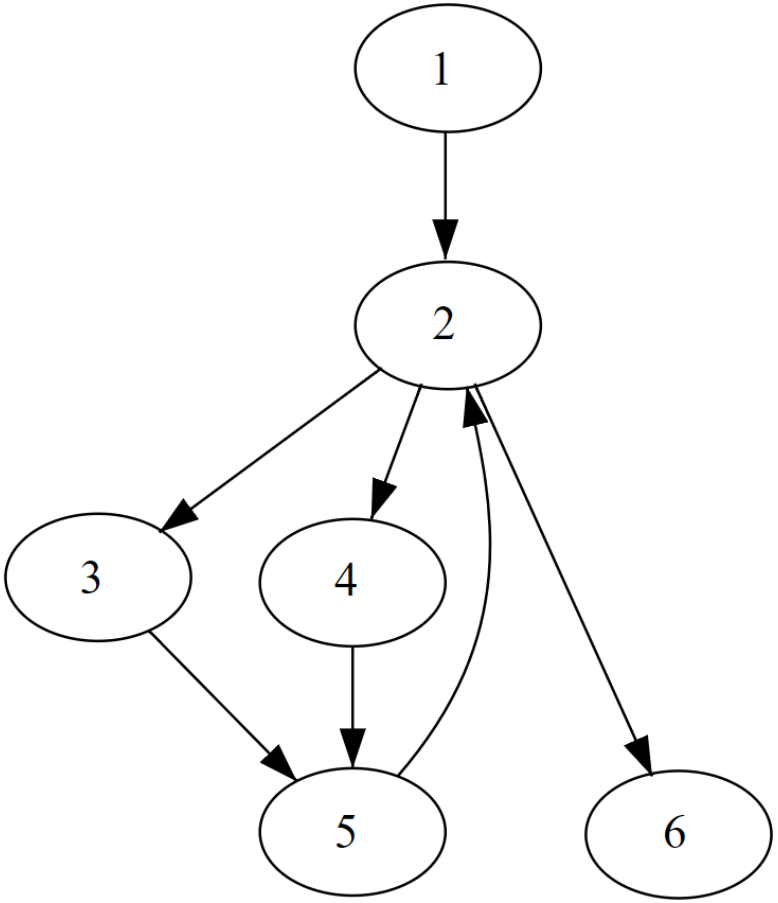
Control flow graph

- The data structure used by the compiler to represent procedures in the program. The vertexes are the base blocks and the edges show the links between the blocks.



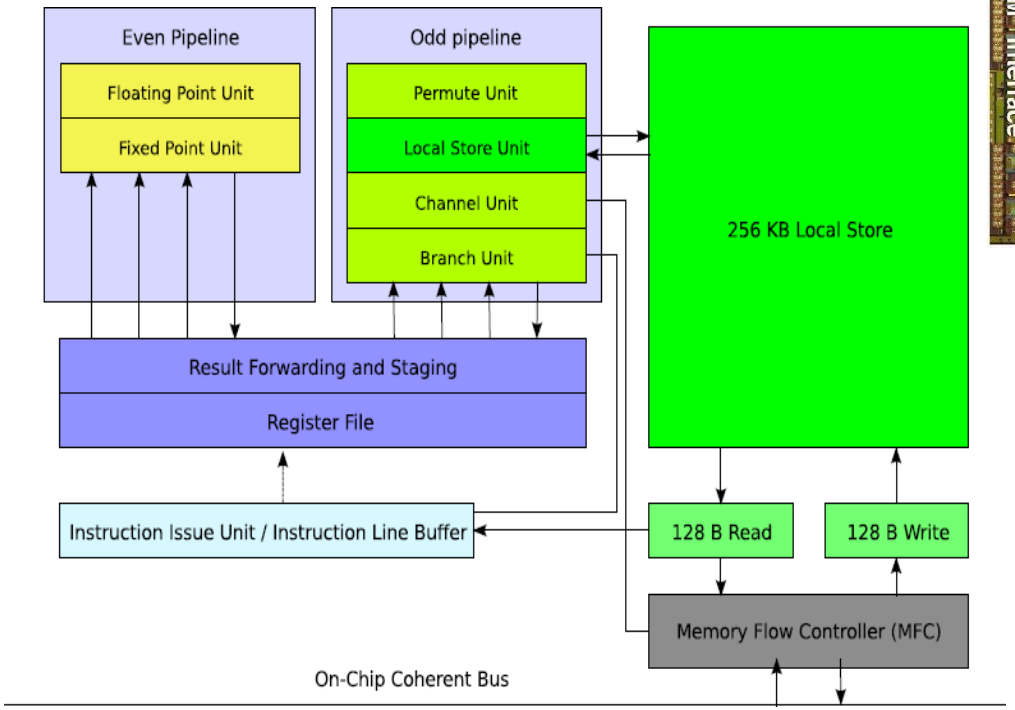


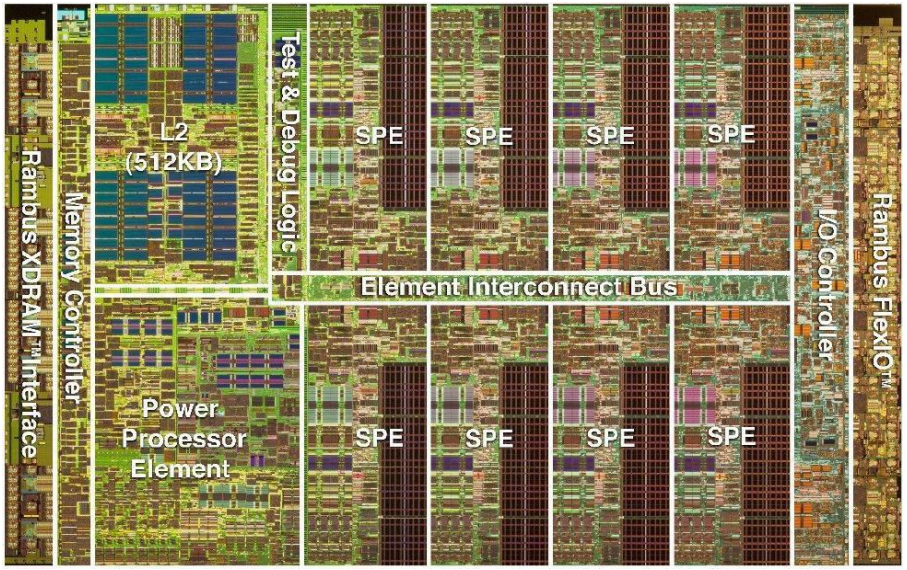
Domination front

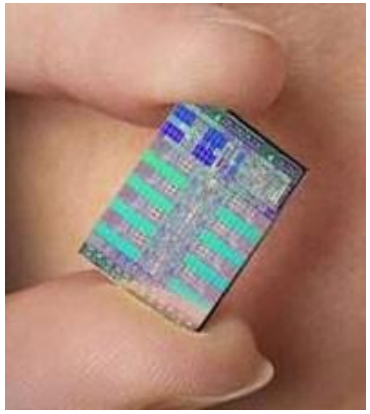


Example

- PowerXCell







Example

- Good code

```
007681 0D  123456                    fma     $118,$124,$67,$118
007681 1D  123456                    lqd     $79,160($53)
007682 0D   234567                   fma     $120,$124,$65,$120
007682 1D   234567                   lqd     $123,176($53)
007683 0D    345678                  fma     $67,$124,$67,$68
007683 1D    345678                  lqd     $121,192($53)
007684 0D     456789                 fma     $68,$124,$69,$70
007684 1D     456789                 lqd     $119,208($53)
007685 0D      567890                fma     $65,$124,$65,$66
007685 1D      567890                lqd     $117,224($53)
007686 0D            678901          fma     $69,$124,$69,$72
007686 1D            678901          lqd     $66,240($53)
```

Example

- Bad code

```
005616 1   678901                        lqd        $54,5568($1)
005622 0    -----2345                    rotmi      $42,$54,-31
005626 0          ---67                  a          $42,$54,$42
005628 0             -8901               rotmai     $52,$42,-1
005632 0               ---23             cgti       $42,$52,0
005634 1                     -4567       brz        $42,.LC___88
005635 1                          5678brz          $27,.LC___88
005636 0D                           67a            $54,$127,$7

006244 0  ----456789                        dfm     $115,$115,$113
006257 0  -------7890123456789              dfm     $115,$115,$44
006264 0          ------4567890123456       fscrrd  $114
006277 0                 ------------       selb    $113,$114,$45,$46
                         78
```

CISC - Complex Instruction Set Computer

- The desire to optimize the pipeline processing led to a change in design of the processors - the transition from CISC to RISC architectures

- CISC architecture - classical processors from 60s and 70s of the XX century
- Complex orders
  - complex addressing modes
  - many addressing modes
  - variable length instruction and significantly different execution time
  - complex fetching from memory and decoding
- a large number of orders (on the processor instructions list)

CISC

- Complex command:
  - fetch two arguments from memory addresses computed by a complex addressing modes, perform the operation, save the result in memory address calculated in a complex manner.
- Complex addressing mode:
  - calculation of the address is based on the: primary address, explicitly specified offset and offset of the corresponding index stored in the appropriate register, which should be multiplied by the scaling factor

CISC

- Pros
  - Facilitation of programming in assembler
  - Reducing the number of orders in the compiled code (minimal requirements for transfer rate of commands and cache size to store them)
- Cons
  - Difficult work for optimizing compilers
  - Complex decoding orders
  - Difficult implementation of pipelining

RISC - Reduced Instruction Set Computer

- RISC revolution of the 80's of the twentieth century
- reducing the number of processor instructions (complex orders converted into sequences of simple commands)
- reducing the number of orders formats
- simple orders and simple addressing modes
- separation of execution orders from the read/write orders
  - load-store architecture
- increase the number of registers

RISC

- Pros
  - High speed of simplified instruction decoding
  - Simplified pipelining
  - The possibility to increase the clock frequency
  - Facilitate the operation of optimizing compilers
- Cons:
  - A large number of commands in the code
  - Need to fast transfer commands from memory (motivation for cache development)

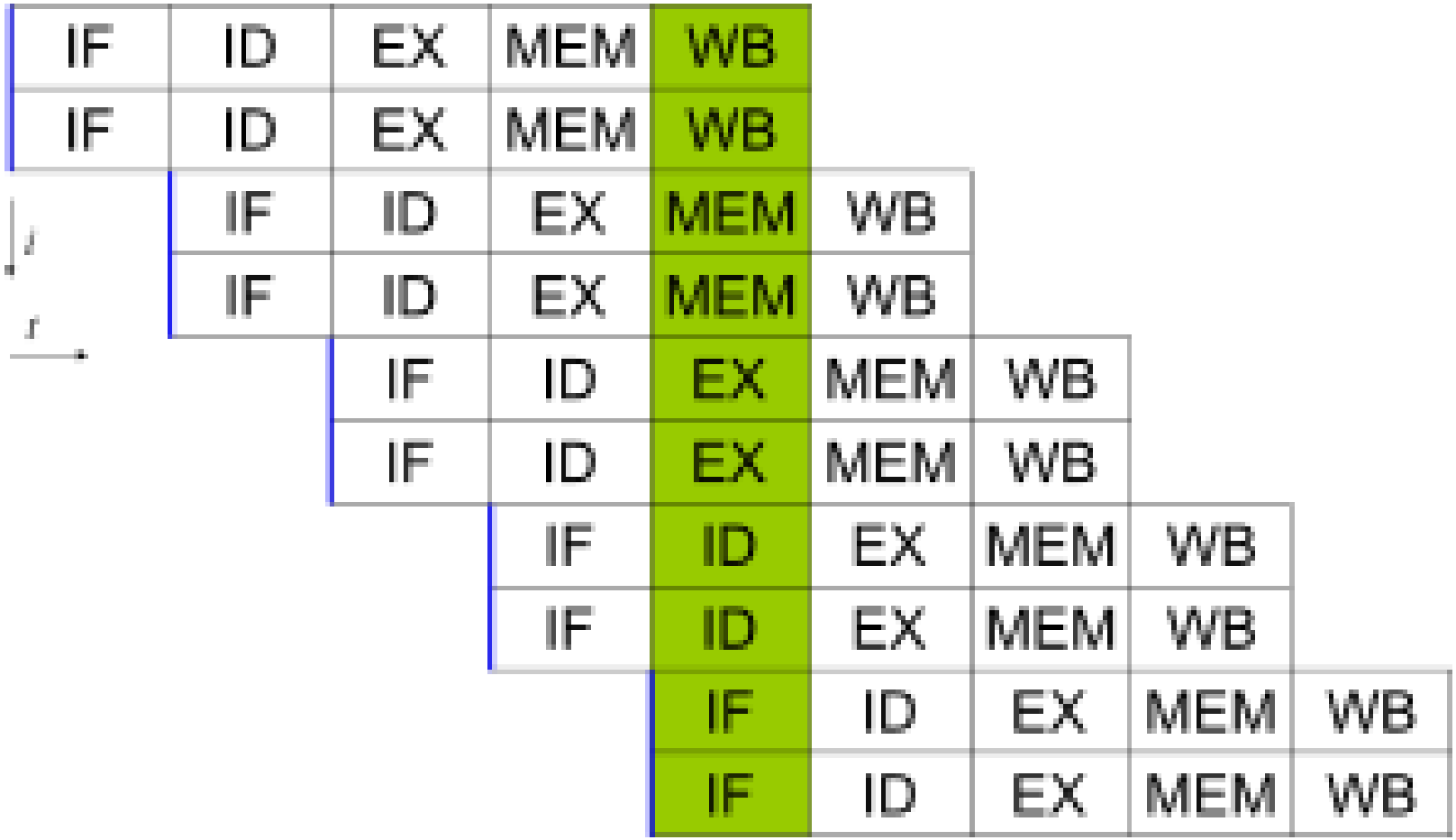
Comparison CISC-RISC

Parameters	CISC	RISC
number of orders	Hundreds	Dozens
maximum length of the order	Tens of bytes	A few bytes
number of order formats	Tens	A few
number of addressing modes	Tens	A few
indirect addressing	Yes	No
the maximum number of arguments	A few	One

Modern processors

- The increase in complexity of microprocessors allows to expand their functionality and speed up
- This is achieved by:
  - Insertion of a many functional units implementing the same stage of the pipelining - superscalar
  - Increasing the number of stages of the stream – super pipelining
  - The use of branch prediction systems
  - Fetch orders in advance (prefetching)
  - Carrying out operations in a different order (out-of-order execution, the pot of dozens of orders processed concurrently)
  - Adding new commands (for example vector = SIMD)
  - Hardware support for multi-threading

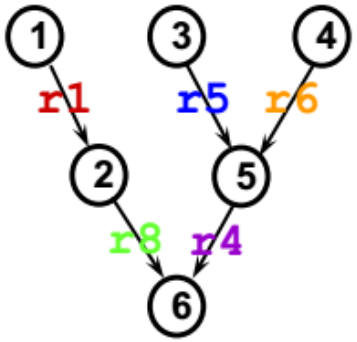
Superscalar



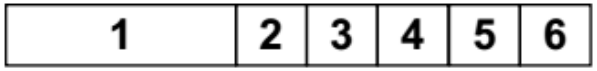
Out-of-order execution

```
(1) r1 ← r4 / r7      /* assume division takes 20 cycles */
(2) r8 ← r1 + r2
(3) r5 ← r5 + 1
(4) r6 ← r6 - r3
(5) r4 ← r5 + r6
(6) r7 ← r8 * r4
```

Data Flow Graph

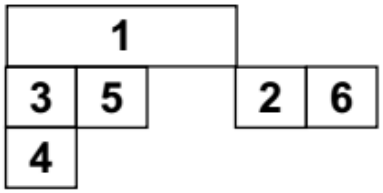


In-order execution

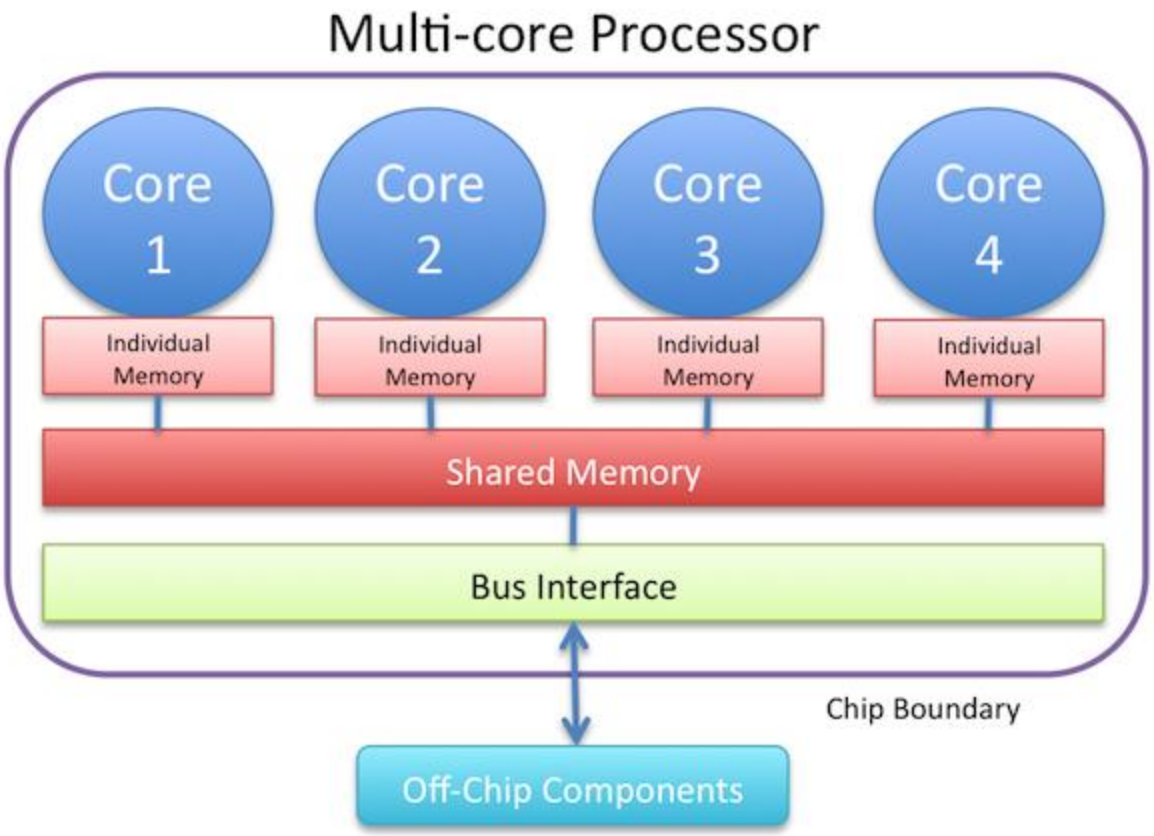


In-order (2-way superscalar)

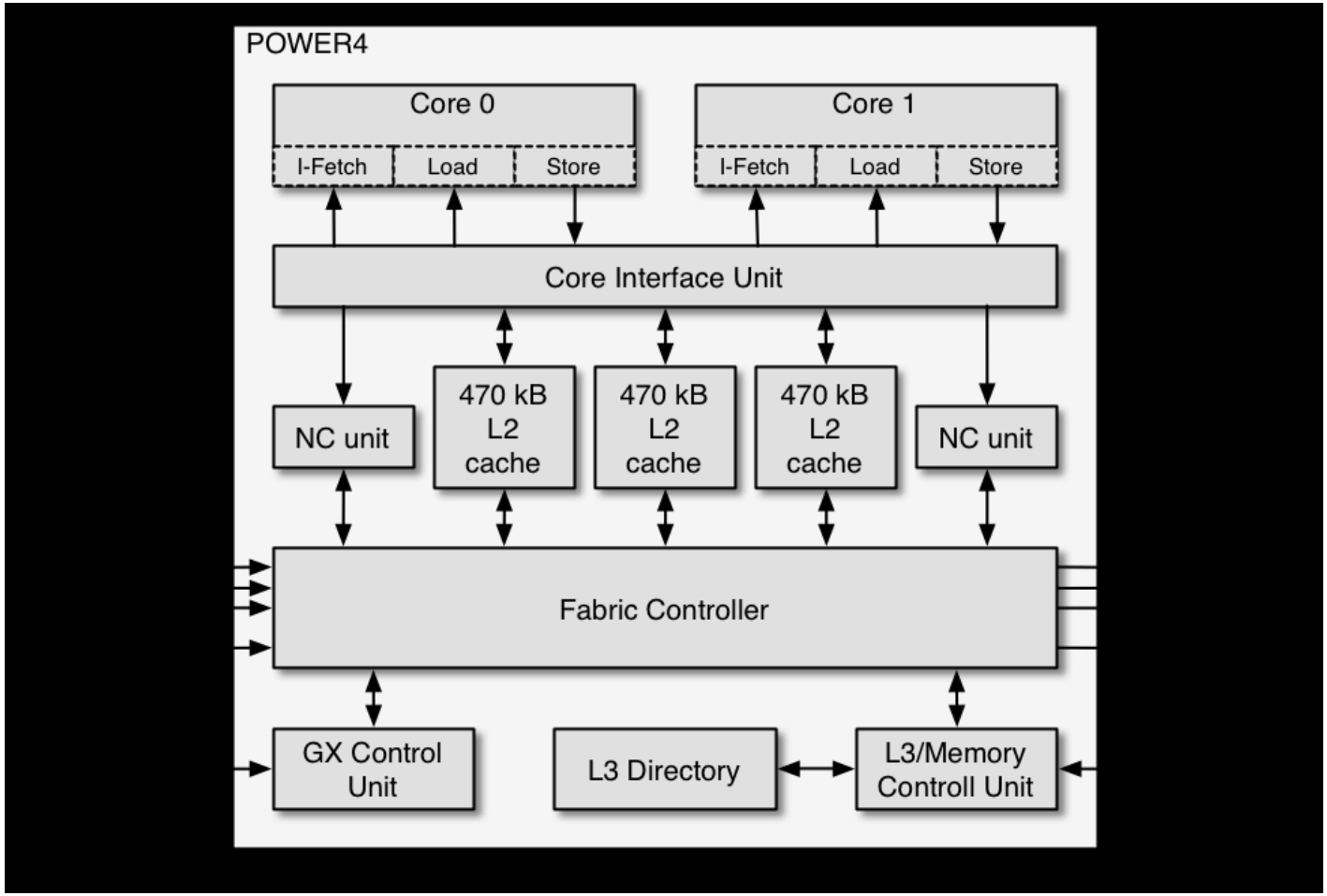
Out-of-order execution



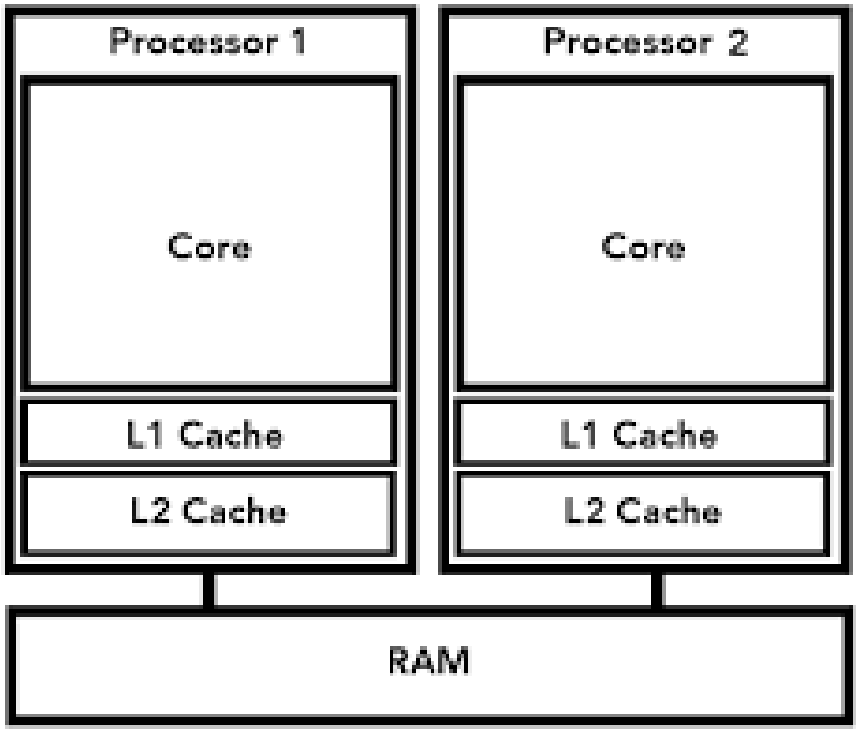
Multicore


Multi-core Processor
Core 1
Core 2
Core 3
Core 4
Individual Memory
Individual Memory
Individual Memory
Individual Memory
Shared Memory
Bus Interface
Chip Boundary
Off-Chip Components

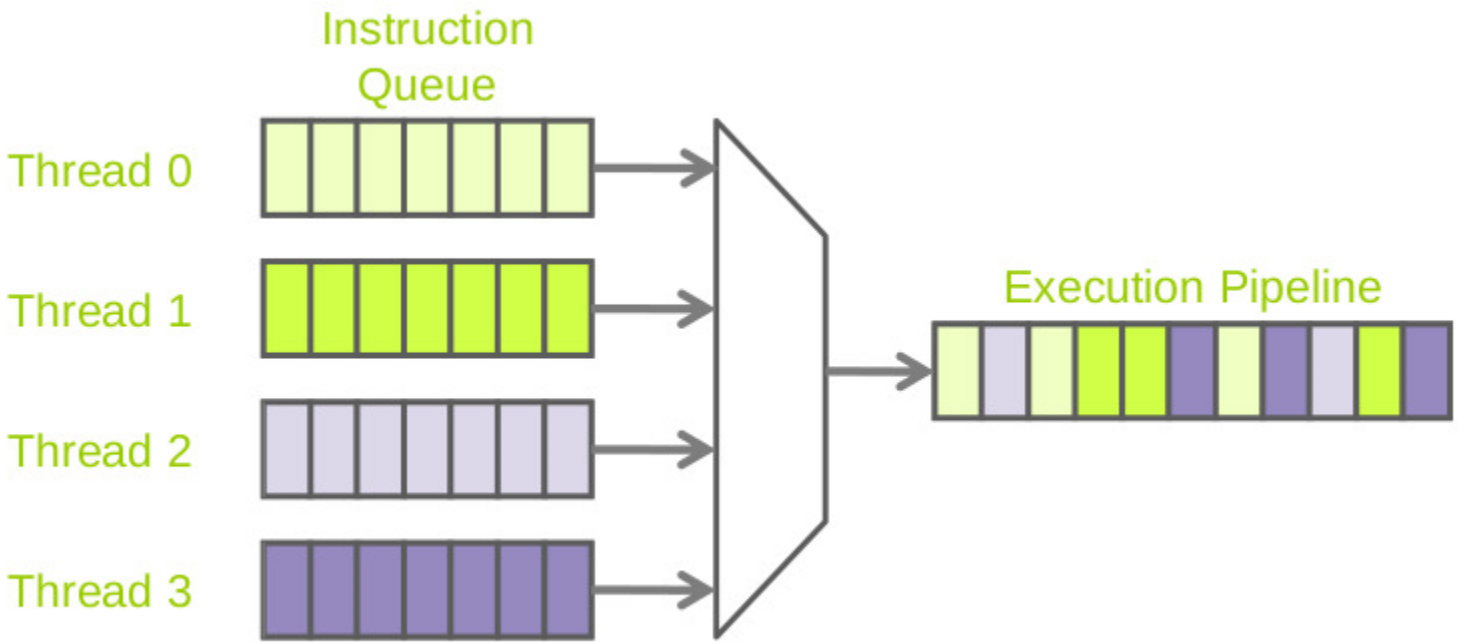
Power4


POWER4
Core 0
I-Fetch
Load
Store
Core 1
I-Fetch
Load
Store
Core Interface Unit
NC unit
470 kB
L2
cache
470 kB
L2
cache
470 kB
L2
cache
NC unit
Fabric Controller
GX Control
Unit
L3 Directory
L3/Memory
Controll Unit

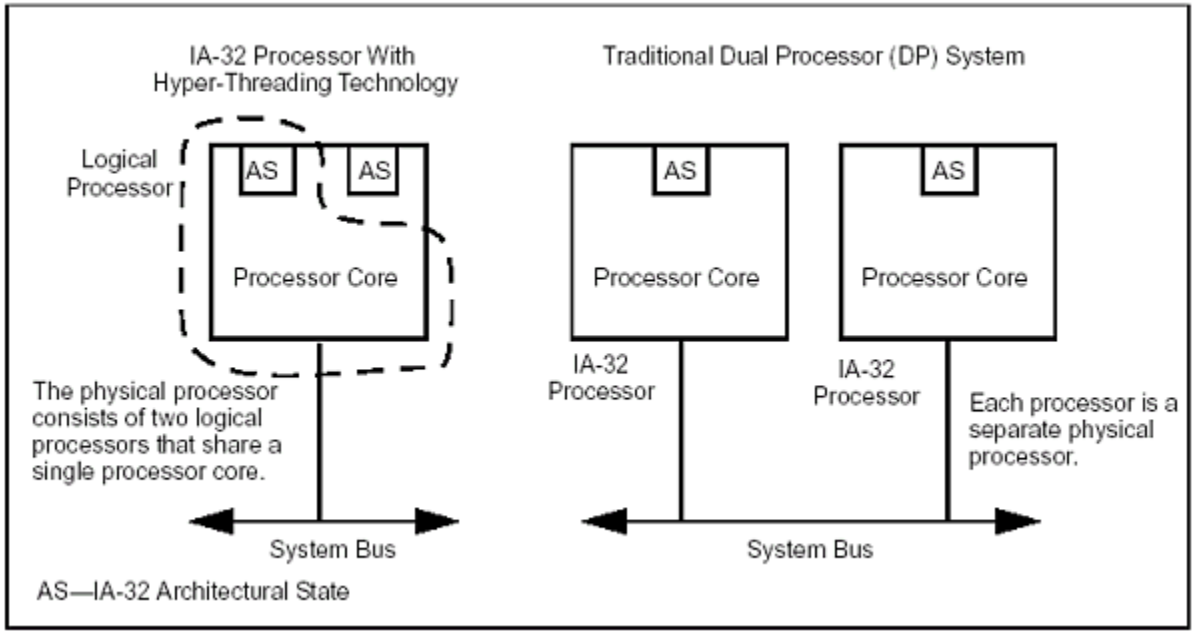
Multi processors



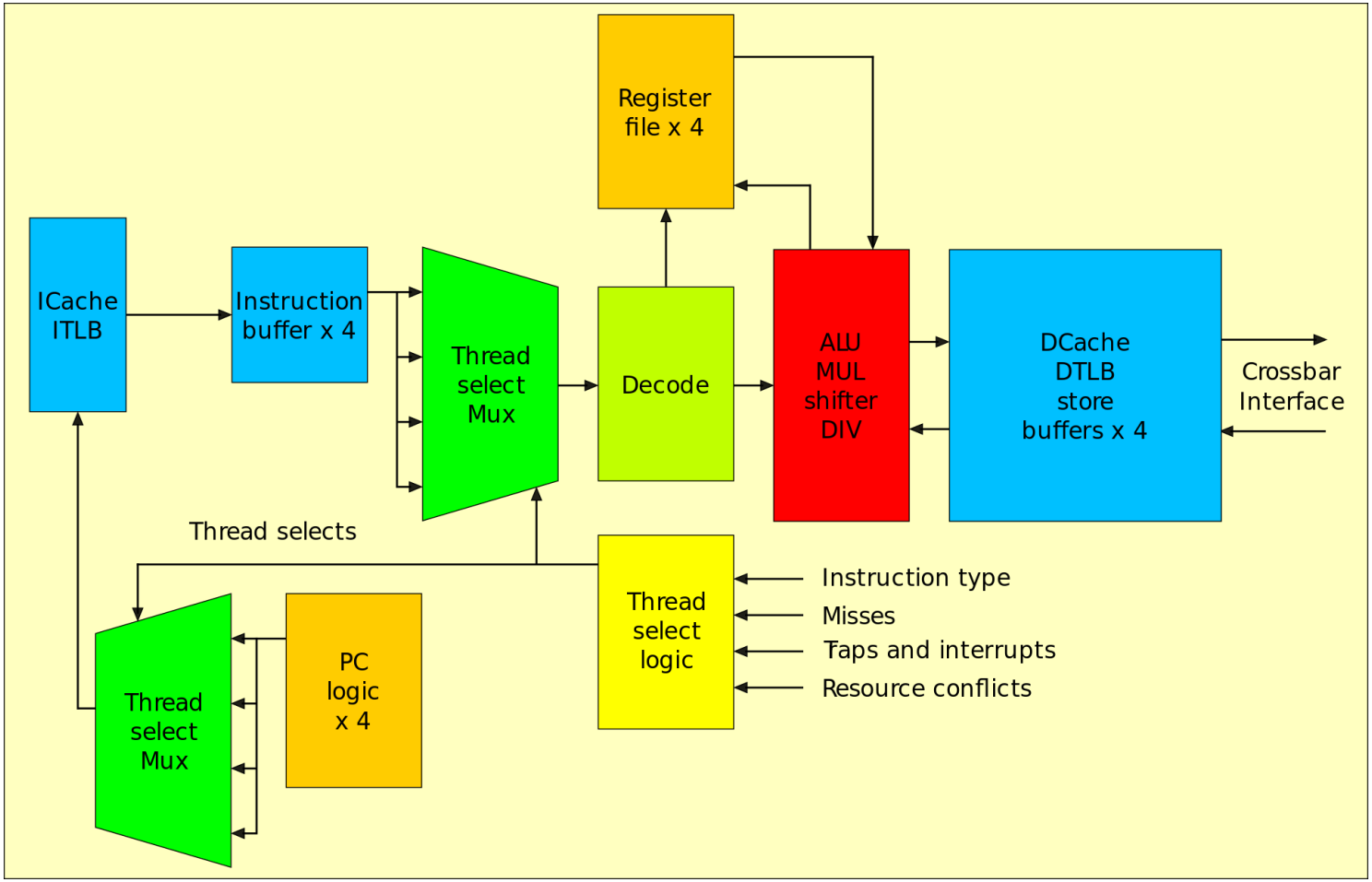
Multithreading



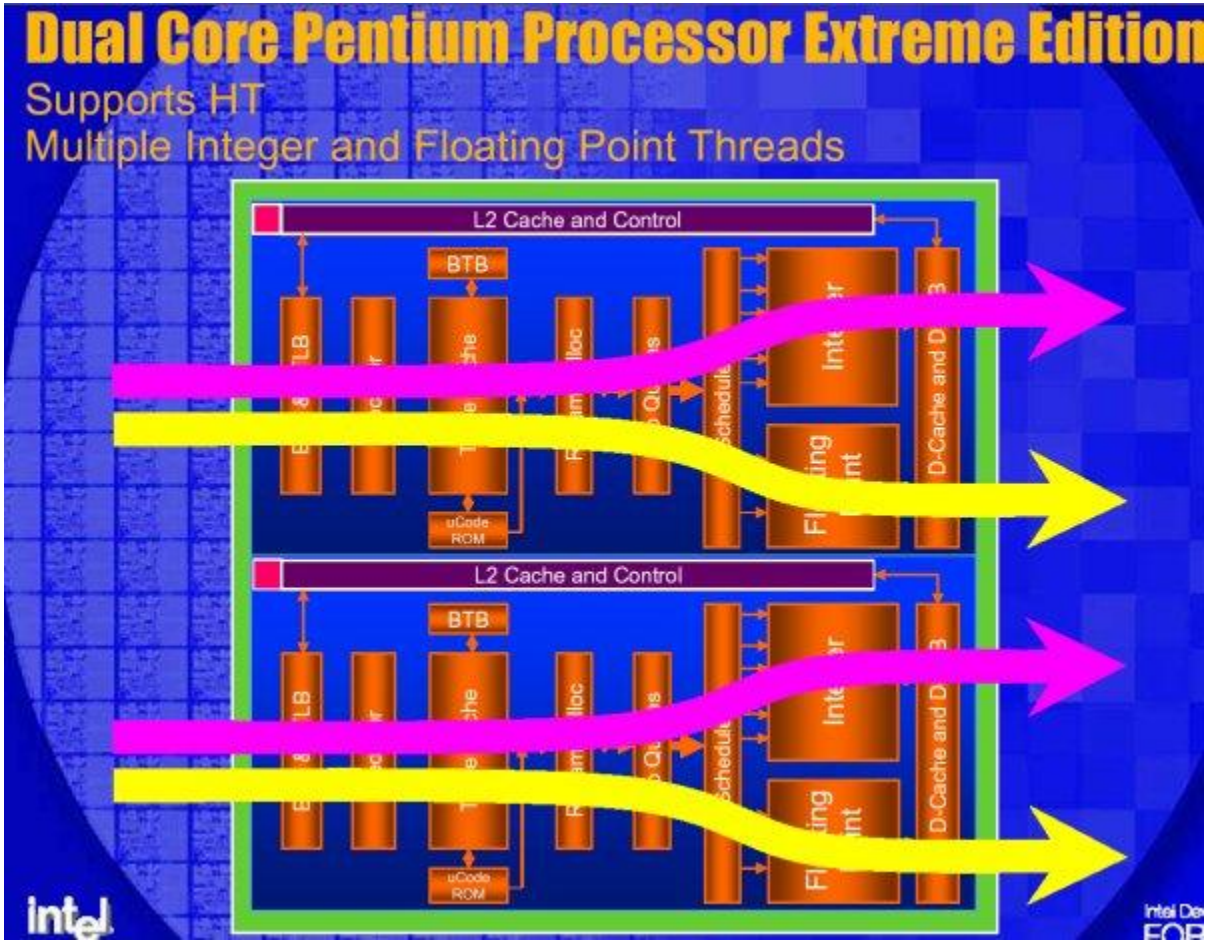
Pentium 4


IA-32 Processor With
Hyper-Threading Technology
Traditional Dual Processor (DP) System
Logical
Processor
AS
AS
Processor Core
AS
Processor Core
AS
Processor Core
IA-32
Processor
IA-32
Processor
The physical processor
consists of two logical
processors that share a
single processor core.
Each processor is a
separate physical
processor.
System Bus
System Bus
AS—IA-32 Architectural State

UltraSparcT1



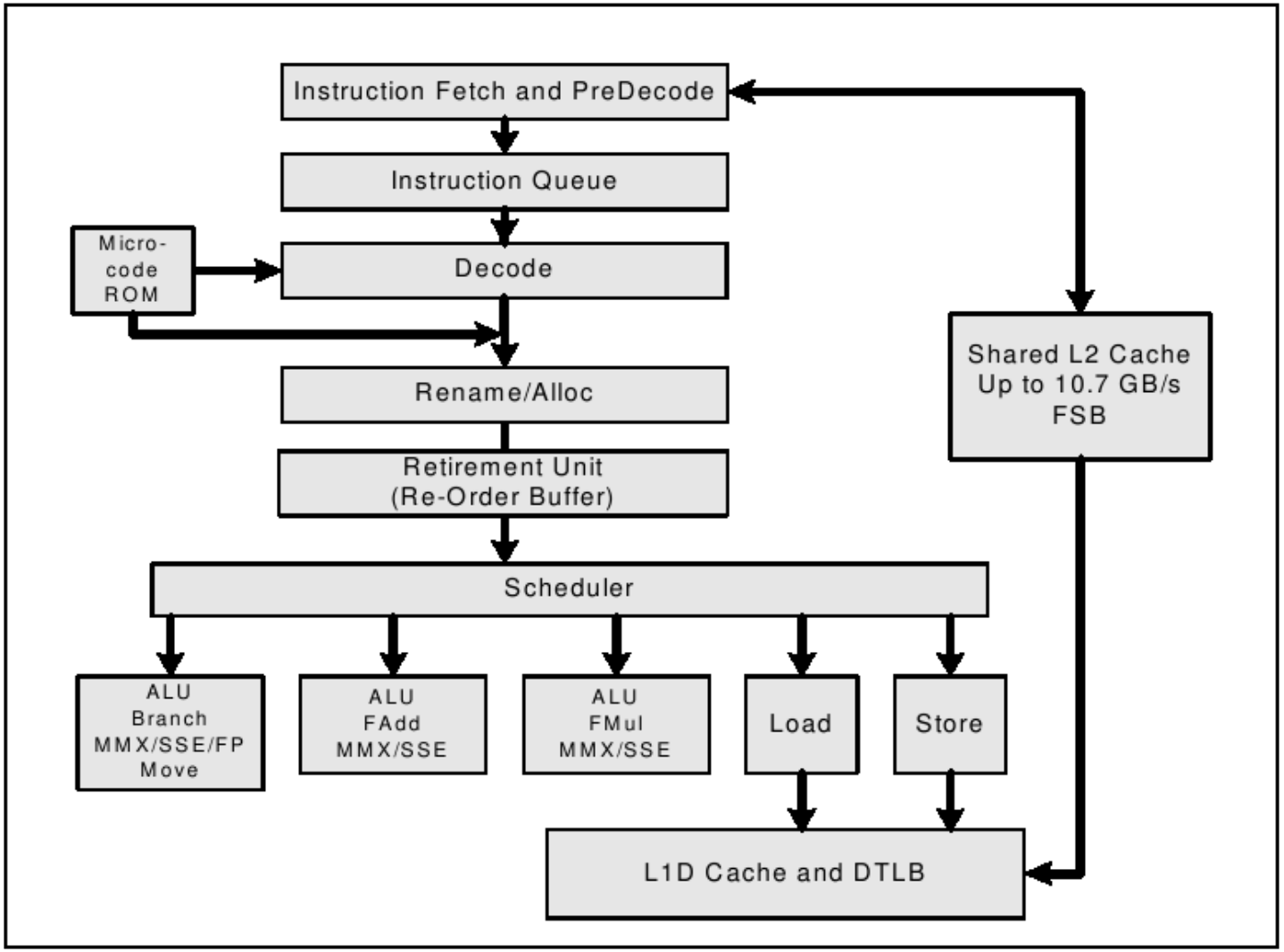
Pentium EE

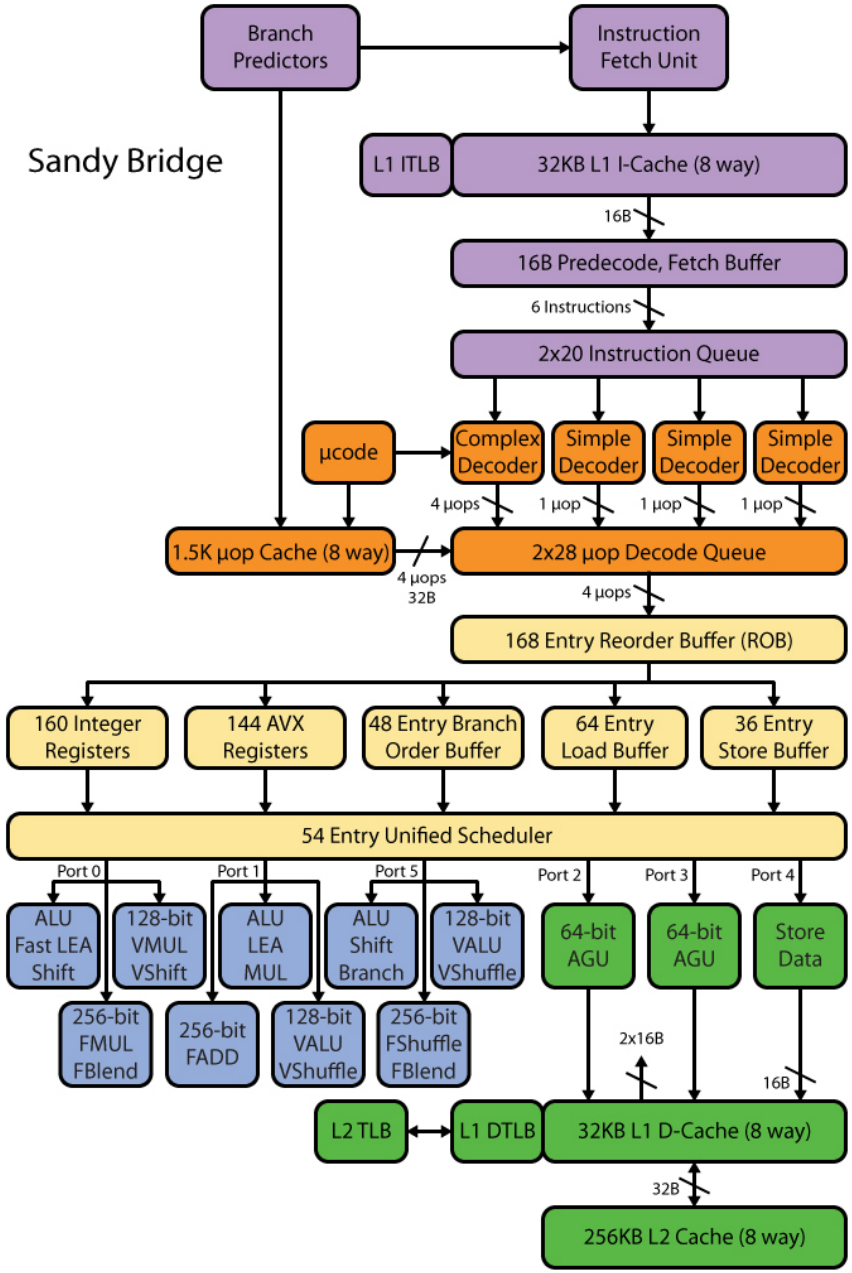

Dual Core Pentium Processor Extreme Edition
Supports HT
Multiple Integer and Floating Point Threads
L2 Cache and Control
BTB
uCode ROM
D-Cache and D
L2 Cache and Control
BTB
uCode ROM
D-Cache and D
intel

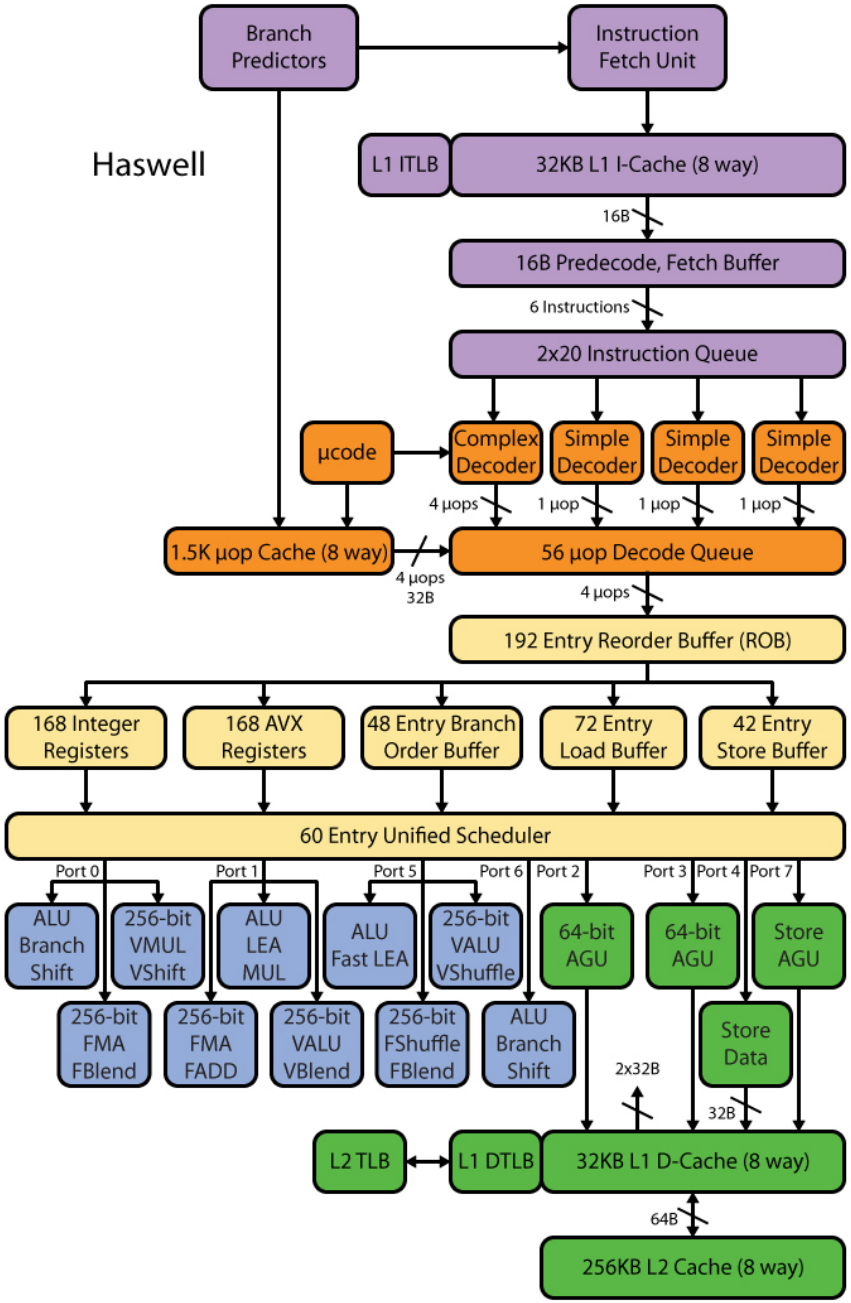
Vector registers

- X86
  - SSE - 128 bit
  - AVX(2) - 256 bit
  - AVX-512 - 512 bit
- ARM
  - NEON - 128 bit
- POWER
  - AltiVec/VMX/VSX - 128 bit
  - QPX - 256 bit
- SPARC
  - HPC-ACE - 128 bit
  - HPC-ACE2 - 256 bit

Intel Core


Instruction Fetch and PreDecode
Instruction Queue
Micro-
code
ROM
Decode
Rename/Alloc
Retirement Unit
(Re-Order Buffer)
Shared L2 Cache
Up to 10.7 GB/s
FSB
Scheduler
ALU
Branch
MMX/SSE/FP
Move
ALU
FAdd
MMX/SSE
ALU
FMul
MMX/SSE
Load
Store
L1D Cache and DTLB


Sandy Bridge
Branch Predictors
Instruction Fetch Unit
L1 ITLB
32KB L1 I-Cache (8 way)
16B
16B Predecode, Fetch Buffer
6 Instructions
2x20 Instruction Queue
µcode
Complex Decoder
Simple Decoder
Simple Decoder
Simple Decoder
4 µops
1 µop
1 µop
1 µop
1.5K µop Cache (8 way)
4 µops
32B
2x28 µop Decode Queue
4 µops
168 Entry Reorder Buffer (ROB)
160 Integer Registers
144 AVX Registers
48 Entry Branch Order Buffer
64 Entry Load Buffer
36 Entry Store Buffer
54 Entry Unified Scheduler
Port 0
Port 1
Port 5
Port 2
Port 3
Port 4
ALU Fast LEA Shift
128-bit VMUL VShift
ALU LEA MUL
ALU Shift Branch
128-bit VALU VShuffle
64-bit AGU
64-bit AGU
Store Data
256-bit FMUL FBlend
256-bit FADD
128-bit VALU VShuffle
256-bit FShuffle FBlend
2x16B
16B
L2 TLB
L1 DTLB
32KB L1 D-Cache (8 way)
32B
256KB L2 Cache (8 way)


Haswell
Branch Predictors
Instruction Fetch Unit
L1 ITLB
32KB L1 I-Cache (8 way)
16B
16B Predecode, Fetch Buffer
6 Instructions
2x20 Instruction Queue
µcode
Complex Decoder
Simple Decoder
Simple Decoder
Simple Decoder
4 µops
1 µop
1 µop
1 µop
1.5K µop Cache (8 way)
4 µops
32B
56 µop Decode Queue
4 µops
192 Entry Reorder Buffer (ROB)
168 Integer Registers
168 AVX Registers
48 Entry Branch Order Buffer
72 Entry Load Buffer
42 Entry Store Buffer
60 Entry Unified Scheduler
Port 0
Port 1
Port 5
Port 6
Port 2
Port 3
Port 4
Port 7
ALU Branch Shift
256-bit VMUL VShift
ALU LEA MUL
ALU Fast LEA
256-bit VALU VShuffle
64-bit AGU
64-bit AGU
Store AGU
256-bit FMA FBlend
256-bit FMA FADD
256-bit VALU VBlend
256-bit FShuffle FBlend
ALU Branch Shift
Store Data
2x32B
32B
L2 TLB
L1 DTLB
32KB L1 D-Cache (8 way)
64B
256KB L2 Cache (8 way)

Performance of the computers

- An attempt to estimate:
  - which processor commands are executed in order to realize tasks (note: different compilers may use different sets of commands!)
  - what is the duration of each of the commands (in the number of cycles) (Note: time of command depends on whether the order was recently used (whether it is decoded in L1), whether the arguments were recently used (they are in the L1, L2, L3 ...) or commands and data are retrieved from the memory of the address in the TLB(Translation Lookaside Buffer), what other commands are executed concurrently by a processor (hazards))
  - differences may be a factor of ten or even more (e.g. in the case of a page fault)